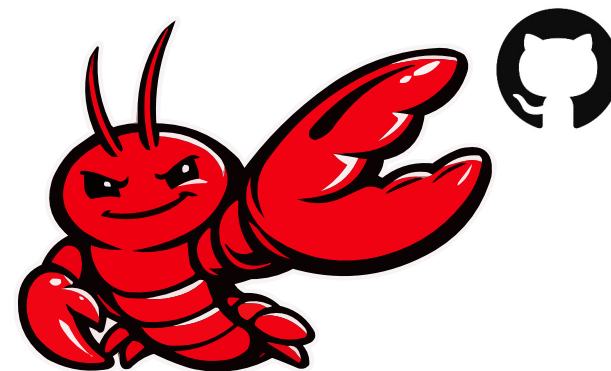


开源项目潜力发现系统

从 **GitHub** 早期信号中挖掘值得观察的开源项目



李文俊 2023150001

陈俊年 2023150108

目标是更早一步——在仓库创建后的早期窗口，识别值得关注的潜力项目

GitHub Trending

看见的是：已经获得关注的项目
问题是：可能错过早期机会

启发

OpenClaw Radar

观察的是：前 30 天的项目行为
输出的是：可追踪的潜力清单

研究支撑：相关研究表明这条路是可行的

Predicting the Popularity of GitHub Repositories

PROMISE '16

stars 时序 + 回归即可预测流行度

→ 流行度是可建模的量

Understanding Factors that Impact Popularity

ICSME '16

2,279 repos · 语言 / 领域显著影响 stars

→ 项目特征带预测信号

What Makes a Popular Academic AI Repository?

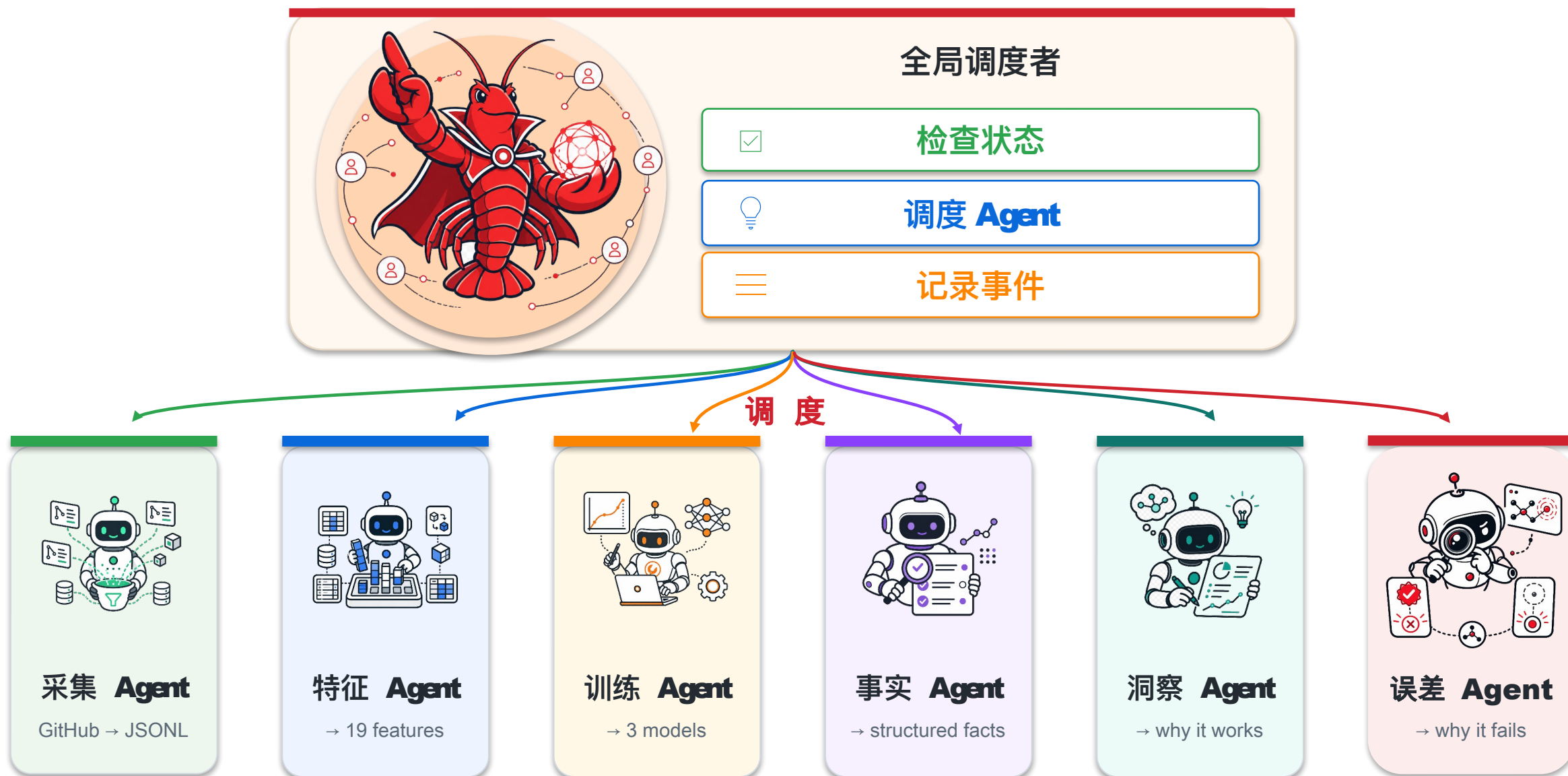
EMSE / ICSE '21

1,149 repos · 11 个特征区分冷热门

→ 早期工程实践可识别

OpenClaw Multiagent : 一个总指挥，六个专职 Agent

3 / 17



每个 **Agent** 都不是一句 **prompt**，而是一套明确的输入、脚本、输出和事件记录

1 采集 **Agent**
repo-collector

GitHub 接口 → 原始记录
产出：仓库数据文件

2 特征 **Agent**
feature-extractor

原始记录 → **19** 个特征
产出：特征表 / 特征顺序

3 训练 **Agent**
model-trainer

特征表 → 模型指标
产出：模型结果文件

4 事实 **Agent**
diagnostic-builder

模型指标 → 结构化事实
产出：事实摘要文件

5 洞察 **Agent**
insight-analysis

事实摘要 → 受约束解释
产出：洞察分析页面

6 误差 **Agent**
error-analyst

预测错误 → **FP / FN** 规律
产出：错误分析页面

第 7 步 report-generator 再把模型结果、聚类、事实诊断、错误分析和洞察统一写入最终 **HTML** 报告

每个仓库拆成两部分：创建后 30 天内能看到的信息，以及一年后的热度标签



预测逻辑：用前 30 天的输入预测一年后是否进入前 20%，两者之间有一年的空白期，模型无法偷看未来

数据样例： 30 天快照是输入，未来标签是答案

用一条仓库记录说明数据切分：模型只能看左边，右边只用来判断是否成功



模型不直接吃原始 **JSON**，而是把 **30** 天内的行为整理成五类可比较信号



$6 + 1 + 4 + 4 + 4$

把不同仓库变成同一张表，
模型才能比较和排序

6	语言信号	Python、JavaScript、Go、Rust、TypeScript、其他「独热编码」
1	归属信号	仓库属于组织账号，还是个人账号
4	活跃信号	Commit、Issue、PR、Contributor
4	文档信号	是否有 README、README 长度、图片、演示链接
4	派生信号	总活跃度、平均提交、PR / Issue、有无 PR

19 个特征覆盖语言、活跃度、文档与协作密度，由同一条 **feature_schema** 锁定——训练和雷达共用一套定义

训练阶段不只是跑分，还会输出 OOF 概率和 KMeans 聚类，为后续错误分析提供输入

- 01

输入特征矩阵

500 个仓库 × 19 个特征
- 02

训练三类模型

逻辑回归 · 随机森林 · XGBoost
- 03

评估排序能力

5 折交叉验证 · AUC · 前 10 命中率
- 04

保存复用产物 + 分析输入

模型文件 · OOF 概率 · 4 类行为模式

三类模型 + 聚类 + 错误分析

- LR

逻辑回归 · 线性、稳定

AUC
0.883
- RF

随机森林 · 非线性、排序

PR-AUC
0.652
- XGB

梯度提升 · 对照模型

P@10
0.80
- KM

KMeans · 4 类行为模式

正例率
62→2%
- EA

错误分析 · OOF 盲区

15 FP
15 FN

指标解释：为什么不能只看一个 AUC

9 / 17

这个任务本质是候选发现，所以要同时看整体排序、正例识别、前排命中和时间泛化

AUC

整体排序能力

热门 vs 冷门配对中
把热门排前面的概率

0.888

**PR-
AUC**

少数正例识别

正例仅约 20% 时
比准确率更能反映识别能力

0.652

P@10

前排候选质量

只看排在最前的 10 个
贴近日常使用场景

0.80

**Time-
Split**

时间泛化

按时间先后切分训练 / 测试
检验是否过于乐观

-0.0048

解释顺序：**AUC** 看排序 · **PR-AUC / P@10** 看候选质量 · **Time-Split** 看时间稳定性

实验结果：早期信号能排序，也能形成可解释的行为分群

模型最有价值的不是“预测每一个项目”，而是把候选项目排到更值得看的顺序



结论：早期 **README** 能继续提升排序；聚类进一步显示，不同行为模式的正例率从 **62%** 到 **2%** 差异明显

用同一套 19 个早期特征，把 是否进前 20% 的分类，扩展为预测约一年后的 $\log(\text{star 数})$

0.50

R^2 (解释方差)

岭回归 **Ridge** · 最佳

RMSE $1.24 \cdot \log_{1+}$

0.48

R^2 (解释方差)

随机森林 **RF**

RMSE $1.27 \cdot \log_{1+}$

0.46

R^2 (解释方差)

XGBoost

RMSE $1.29 \cdot \log_{1+}$

两个互补视角

分类回答会不会进前 20% (AUC 0.888) · 回归回答能火到多大 ($R^2 \approx 0.50$) ——同一套信号，两个互补答案

结论：早期行为不仅能区分头部项目，还能在一定程度上预测它们最终的热度量级

单次问大模型也能写分析；OpenClaw 解决的是多步骤任务如何稳定运行、留下证据、复盘模型盲区

直接问大模型

- ❌ 把结果文件丢给模型总结
- ❌ 可能引用不存在的数字
- ❌ 每次回答不一定一致
- ❌ 难以接入完整流程

OpenClaw workflow

- ✅ 技能固定输入 / 输出契约
- ✅ 先算事实摘要，再定位 FP/FN
- ✅ 引用数字必须能在事实里校验
- ✅ 控制台展示状态与产物

先算事实，
再让模型解释

结果文件



事实诊断 → 结构化事实



洞察 / 误差 → 受约束解释

OpenClaw 把模型训练、聚类分析、错误案例和大模型解释组织成可审计、可复现的多智能体流程

聚类 + 错误分析：不仅看分数，也看模型盲区

error-analyst 把模型的高置信错误单独拿出来分析，让结果从 " 跑分 " 变成 " 可复盘 "

4 类

KMeans 行为模式

62%

最高正例率 cluster

15

高置信假阳性 FP

15

低估但成功 FN

行为分群：同样是早期项目，成功率差异很大

错误分析：高分不一定成功，低分也可能爆发

聚类	仓库数	正例率	行为特征
Cluster 0	89 个	62%	中等活跃 · 文档完整
Cluster 1	62 个	40%	高提交 · 高 PR
Cluster 2	252 个	8%	低协作 · 低互动
Cluster 3	97 个	2%	几乎无 issue / PR

FP	高活跃但未成功 平均 28 提交 · 2 Issue → 最终 ~28 星
FN	早期低调，后期爆发 平均 26 提交 · 0 Issue → 最终 ~492 星

模型的真正价值：不仅排序候选项目，还能定位哪些信号会误导模型、哪些潜力项目可能被遗漏

把 OpenClaw 的结构化事实摘要喂给三个旗舰大模型，自动校验它们引用的每个数字 ($\pm 10\%$ 容差)

98.6%

数字准确率

DeepSeek V4 Pro

146 引用 · 2 处偏差

100%

数字准确率

Kimi K2

77 引用 · 0 偏差

100%

数字准确率

Qwen3.7 Max

108 引用 · 0 偏差

为什么这页能证明 **OpenClaw** 的价值

先算事实、再解释、逐数字校验——Kimi / Qwen 100% 准确，DeepSeek 引用最多但有 2 处偏差，几乎零编造

结论：把分析锚定在已验证的事实上，能直接量化、对比不同大模型的可靠性，而不是让它们自由发挥

应用层不重新发明模型，而是复用同一套 19 个特征、训练好的模型文件、动态阈值和错误分析经验

01

近期仓库

创建 30-45 天

02

特征对齐

同一套 19 个特征

03

模型打分

复用训练好的随机森林，输出 0-1 潜力分数

04

行动分层

按训练集分位数划分

实时扫描结果 — 模型潜力分数排名

1	lunashia/o-m_beatmap_trainer	Python	0.935
2	Kavishp7499/qp	Go	0.718
3	NotHarshhaa/mlflow-k8s-operator	Go	0.718
4	esengine/DeepSeek-Reasonix	Go	0.718
5	eSlider/bonsai-ollama	Go	0.718

输出今日观察清单，后续可回看 star 增长

分类 + 回归 + **OpenClaw** workflow + 错误分析 + 大模型对比

模型层

早期信号可以排序潜力仓库

AUC 0.888 · PR-AUC 0.652

分析层

聚类 and 错误分析让模型盲区可复盘

4 类行为模式 · 15 FP · 15 FN

应用层

Today Radar 复用同一套特征和阈值

19 个特征 · 动态阈值

核心逻辑：早期信号可排序 → 聚类揭示模式 → 错误分析定位盲区 → **Today Radar** 落地为可运行系统



Thank You

OpenClaw · 开源项目潜力发现系统

李文俊 **2023150001** · 陈俊年 **2023150108**

github.com/WenjunLi2004/DataMining---OpenClaw